

Progressão

O personagem melhora depois de cada vitória. Três métodos pequenos, e um quarto que junta os três.

NESTA AULA

1. Recapitular onde vocês pararam
2. Construir `melhorar_ataque` e `melhorar_defesa`
3. Construir `melhorar_vida_maxima`
4. Juntar os três em `subir_nivel`
5. Chamar `subir_nivel` depois de cada vitória no `main.py`

Antes de começar, confira a sua pasta:

ARQUIVO	O QUE TEM DENTRO
<code>personagem.py</code>	Personagem, <code>_vida</code> , <code>_vida_maxima</code> , <code>curar</code> , <code>ataque</code> , <code>defesa</code>
<code>itens.py</code>	Item, PocaoDeVida, Bomba, PocaoDeForca, AnelDeProtecao, Inventario
<code>classes.py</code>	Guerreiro, Mago, Goblin, Orc, Troll, Dragao
<code>main.py</code>	Fase, a lista <code>fases</code> , o menu, o combate

Onde vocês pararam

O jogo já tem fases, cada uma com um inimigo e um prêmio. O laço principal, no `main.py`, é este:

```
for fase in fases:
    print("\n==", fase.nome, "==")
    print(fase.descricao)

    venceu = combate(jogador, fase.inimigo)
    if not venceu:
        print("Fim de jogo.")
        return
    jogador.curar(50)
    print(jogador.nome, "descansa e recupera 50 de vida")

    jogador.inventario.adicionar(fase.premio)
    print(fase.inimigo.nome, "deixou cair:", fase.premio)
```

Repare na linha logo depois do `if not venceu`. É exatamente ali, entre vencer a fase e descansar, que a progressão de hoje vai entrar. Uma linha nova, nada mais.

No fim da aula passada, ficou esta promessa, sem explicação nenhuma ainda:

```
def melhorar_ataque(self, quantidade):
    self.ataque += quantidade
```

Hoje essa linha ganha uma explicação, dois métodos irmãos, e um lugar para morar.

VOCABULÁRIO RÁPIDO, SE ALGUÉM ESQUECEU

`self.ataque` / `self.defesa` — atributos comuns do `Personagem`, sem `_` na frente. Qualquer método da classe pode mexer neles direto.

`self._vida_maxima` — atributo **privado**. De fora da classe só se lê, através da `property` `vida_maxima`. De dentro da classe, se muda direto.

nível (novo hoje) — um contador simples de quantas vezes o personagem já melhorou.

progressão (novo hoje) — o personagem que termina o jogo mais forte do que começou, sem o jogador precisar fazer nada além de vencer fases.

melhorar_ataque e melhorar_defesa

Os dois métodos mais simples de hoje. Cada um mexe em exatamente um número do personagem.

O QUE CADA UM PRECISA FAZER

- `melhorar_ataque(self, quantidade)` — soma `quantidade` em `self.ataque`.
- `melhorar_defesa(self, quantidade)` — soma `quantidade` em `self.defesa`.
- Nenhum dos dois imprime nada. Quem chama decide se quer avisar o jogador.

```
class Personagem:
    ...
    def melhorar_ataque(self, quantidade):
        self.ataque += quantidade

    def melhorar_defesa(self, quantidade):
        self.defesa += quantidade
```

QUANTO CÓDIGO ISSO É

Duas linhas cada. São os métodos mais curtos que vocês vão escrever hoje — e servem de base para o terceiro, que é só um pouco maior.

O que muda, o que fica igual

melhorar_ataque

```
def melhorar_ataque(  
    self, quantidade):  
    self.ataque += quantidade
```

melhorar_defesa

```
def melhorar_defesa(  
    self, quantidade):  
    self.defesa += quantidade
```

● muda no nome do método ● muda no atributo somado

A ÚNICA DIFERENÇA, E SÓ ESSA

O nome do método e o atributo que ele soma. A assinatura `(self, quantidade)`, o corpo de uma linha, tudo o resto é idêntico. É o mesmo padrão que `ataque += self.forca` e `defesa += self.protecao` usavam ontem dentro dos itens — só que agora o método mora dentro do próprio `Personagem`, em vez de dentro de um item que é usado nele.

Testando os dois

```
h = Guerreiro("Thoric")  
print("ataque:", h.ataque, "| defesa:", h.defesa)  
h.melhorar_ataque(4)  
h.melhorar_defesa(2)  
print("ataque:", h.ataque, "| defesa:", h.defesa)
```

SAÍDA

```
ataque: 18 | defesa: 8  
ataque: 22 | defesa: 10
```

Por que ataque não tem teto

`curar`, que vocês já conhecem, é cuidadoso: usa `min(...)` para nunca deixar a vida passar da `vida_maxima`.

```
def curar(self, quantidade):  
    if quantidade <= 0:  
        return  
    self._vida = min(self._vida + quantidade, self._vida_maxima)
```

`melhorar_ataque` não tem esse cuidado. Ele só soma. Não existe um `ataque_maximo` no jogo — e por enquanto, de propósito, não vai existir.

VALE PENSAR

Por que `vida` precisa de um teto e `ataque` não? A resposta tem a ver com o que cada número representa: vida é um recurso que se gasta e se recupera dentro dos limites do personagem; ataque é uma força que só cresce. Se um dia vocês quiserem um jogo balanceado, essa pergunta volta — hoje ela fica só na cabeça de vocês.

melhorar_vida_maxima

Vida é diferente de ataque e defesa: ela tem um teto, `_vida_maxima`, e um valor atual, `_vida`, que nunca passa do teto.

Se `melhorar_vida_maxima` só aumentasse o teto, o personagem levaria um "level up" e continuaria com a mesma vida atual de antes — só que agora com mais espaço vazio acima dela. Não parece uma melhora de verdade. Por isso o método também cura o personagem no mesmo tanto que o teto sobe.

O QUE ELE PRECISA FAZER

- Somar `quantidade` em `self._vida_maxima`.
- Somar `quantidade` também em `self._vida` — o personagem sente a melhora na hora.

```
class Personagem:  
    ...  
    def melhorar_vida_maxima(self, quantidade):  
        self._vida_maxima += quantidade  
        self._vida += quantidade
```

Repare no `_` antes de `vida_maxima` e antes de `vida`. Isso não é acidente.

Mexendo direto no que é privado

`vida_maxima` é uma `property` — vocês já viram isso na aula de encapsulamento. De **fora** da classe, ela só permite leitura:

```
@property
def vida_maxima(self):
    return self._vida_maxima
```

Não existe um `@vida_maxima.setter`. É por isso que `melhorar_vida_maxima` escreve `self._vida_maxima`, com `_`, e não `self.vida_maxima`. Por **dentro** da classe, o atributo real sempre esteve ali, disponível — a `property` só bloqueia quem está de fora.

ERRO COMUM DE COPIAR E COLAR

Quem copia `melhorar_ataque` e troca só o nome, sem prestar atenção, escreve isto:

```
def melhorar_vida_maxima(self, quantidade):
    self.vida_maxima += quantidade # sem o _
    self.vida += quantidade       # sem o _
```

E o programa quebra, alto e na hora — o que, aqui, é uma sorte:

```
AttributeError: property 'vida_maxima' of 'Personagem'
object has no setter
```

Isso é a `property` fazendo exatamente o trabalho para o qual ela existe: qualquer código, de dentro ou de fora, é impedido de sobrescrever o valor sem passar pela regra da classe. Confirmam o `_` nas duas linhas.

Testando

```
h = Guerreiro("Thoric")
h.receber_dano(40)
print("antes:", h.vida, "/", h.vida_maxima)
h.melhorar_vida_maxima(20)
print("depois:", h.vida, "/", h.vida_maxima)
```

SAÍDA

antes: 88 / 120
depois: 108 / 140

Três métodos, um padrão só

Assim como ontem a Poção de vida, a Poção de força e o Anel de proteção compartilhavam a mesma forma por dentro do `usar`, hoje os três métodos de melhora compartilham a mesma forma: recebem um número e somam ele em algum lugar do personagem.

MÉTODO	ATRIBUTO	EFEITO
<code>melhorar_ataque</code>	<code>ataque</code>	<code>self.ataque += quantidade</code>
<code>melhorar_defesa</code>	<code>defesa</code>	<code>self.defesa += quantidade</code>
<code>melhorar_vida_maxima</code>	<code>_vida_maxima</code> e <code>_vida</code>	<code>self._vida_maxima += q</code> / <code>self._vida += q</code>

O PONTO DA AULA, ATÉ AQUI

Nenhum dos três métodos imprime nada, pede confirmação, ou decide sozinho quanto vale a melhora. Eles só sabem **fazer** a melhora quando alguém pede. Quem decide o "quanto" e o "quando" é outro método — o próximo.

subir_nivel

Três métodos separados são flexíveis, mas ninguém quer, toda vez que o jogador vencer uma fase, escrever três linhas soltas chamando cada um com números diferentes.

`subir_nivel` junta os três num só lugar, que decide os valores de uma vez por todas.

Antes do método, uma linha nova no `__init__` — o personagem precisa de um contador:

Já existe

```
def __init__(self, nome, vida,
              ataque, defesa,
              pocoes):
    self.nome = nome
    self._vida = vida
    self._vida_maxima = vida
    self.ataque = ataque
    self.defesa = defesa
    self.defendendo = False
    ...
```

Com a linha nova

```
def __init__(self, nome, vida,
              ataque, defesa,
              pocoes):
    self.nome = nome
    self._vida = vida
    self._vida_maxima = vida
    self.ataque = ataque
    self.defesa = defesa
    self.defendendo = False
    self.nivel = 1
    ...
```

Agora o método:

```
def subir_nivel(self):
    self.nivel += 1
    self.melhorar_ataque(4)
    self.melhorar_defesa(2)
    self.melhorar_vida_maxima(20)
    print(self.nome, "subiu para o nível", self.nivel)
```

`subir_nivel` não soma nada diretamente em `self.ataque`, `self.defesa` ou `self._vida_maxima`. Ele chama os três métodos que já sabem fazer isso. Se um dia vocês quiserem mudar quanto vale um nível, existe **um** lugar para editar — não três.

QUANTO CÓDIGO ISSO É

Cinco linhas. É o método mais comprido de hoje, e mesmo assim ele não sabe *como* cada atributo muda — só sabe *quanto* e *quando*. Quem sabe o "como" continua sendo os três métodos de ontem... digo, os três de hoje mesmo.

Testando

```
h = Guerreiro("Thoric")
print("nível:", h.nível, "| ataque:", h.ataque,
      "| defesa:", h.defesa, "| vida máxima:", h.vida_maxima)
h.subir_nivel()
print("nível:", h.nível, "| ataque:", h.ataque,
      "| defesa:", h.defesa, "| vida máxima:", h.vida_maxima)
```

SAÍDA

```
nível: 1 | ataque: 18 | defesa: 8 | vida máxima: 120
Thoric subiu para o nível 2
nível: 2 | ataque: 22 | defesa: 10 | vida máxima: 140
```

Onde subir_nivel é chamado

Uma linha nova, no `main.py`, dentro do laço de fases — logo depois de confirmar a vitória, antes do descanso:

```
for fase in fases:
    print("\n==", fase.nome, "==")
    print(fase.descricao)

    venceu = combate(jogador, fase.inimigo)
    if not venceu:
        print("Fim de jogo.")
        return

    jogador.subir_nivel()

    jogador.curar(50)
    print(jogador.nome, "descansa e recupera 50 de vida")

    jogador.inventario.adicionar(fase.premio)
    print(fase.inimigo.nome, "deixou cair:", fase.premio)
```

Nada além dessa linha muda no `main.py`. O laço não sabe quanto vale um nível, não sabe que existem três métodos por trás de `subir_nivel` — ele só sabe que, depois de uma vitória, o personagem melhora.

E SE A ORDEM FOR TROCADA?

Se `jogador.curar(50)` vier **antes** de `jogador.subir_nivel()`, o jogo ainda funciona — mas o personagem cura 50 primeiro, com a vida máxima antiga, e só depois ganha os 20 pontos extras de `melhorar_vida_maxima`. O resultado final da vida é igual, mas se algum dia a cura do descanso virar uma porcentagem da vida máxima, a ordem passa a importar de verdade. Chamar `subir_nivel` primeiro é o hábito mais seguro.

Testando a progressão

Jogando do início ao fim com o Guerreiro Thoric, vencendo as quatro fases em ordem, os números de `mostrar_status` (por enquanto ainda sem o nível — isso é o exercício 6) evoluem assim:

FASE VENCIDA	NÍVEL	ATAQUE	DEFESA	VIDA MÁXIMA
— (início)	1	18	8	120
Floresta Sombria	2	22	10	140
Trilha da Montanha	3	26	12	160
Ponte Quebrada	4	30	14	180
Covil do Dragão	5	34	16	200

O Dragão Ancião, a última fase, é o inimigo mais forte do jogo — mas quem chega até ele já não é o mesmo personagem que começou na Floresta Sombria. É essa a ideia inteira de progressão: a dificuldade do jogo fica combinada com o crescimento de quem joga.

Três métodos pequenos, um que os junta

Na próxima aula: alguns bônus não deveriam durar para sempre — o começo dos efeitos temporários.

PRÉVIA DA PRÓXIMA AULA

```
def iniciar_turno(self):  
    self.defendendo = False  
    # ... e se um bônus tivesse que "acabar" aqui?
```

- ✓ `melhorar_ataque` e `melhorar_defesa` existem e cada um soma um número no atributo certo
- ✓ `melhorar_vida_maxima` existe e soma nos dois: `_vida_maxima` e `_vida`
- ✓ `subir_nivel` existe, soma 1 em `self.nivel` e chama os três métodos acima
- ✓ O `__init__` do `Personagem` cria `self.nivel = 1`
- ✓ `main.py` chama `jogador.subir_nivel()` depois de cada vitória

Progressão

Seis passos curtos: três métodos de melhora, o método que os junta, o gancho no jogo, e um retoque final no status.

DEPOIS DA EXPLICAÇÃO DE HOJE.

Este documento não repete o código já mostrado. Ele diz o que construir — quem escreve é você.

Onde você está

Você acabou de ver, na explicação, como `melhorar_ataque`, `melhorar_defesa` e `melhorar_vida_maxima` funcionam, e como `subir_nivel` junta os três. Agora é a sua vez de escrever tudo, sozinho, no seu próprio `personagem.py` e `main.py`.

Confira que você tem isto antes de seguir:

ARQUIVO	O QUE PRECISA TER
<code>personagem.py</code>	Personagem com <code>ataque</code> , <code>defesa</code> , <code>_vida</code> , <code>_vida_maxima</code> e as properties <code>vida</code> / <code>vida_maxima</code> .
<code>main.py</code>	A classe <code>Fase</code> , a lista <code>fases</code> e o laço principal já funcionando.

SE O SEU JOGO NÃO ESTIVER NESSE PONTO

Não perca a aula de hoje consertando a de ontem. Pegue os quatro arquivos prontos em `Codigo Base/`, na mesma pasta deste PDF, e comece os exercícios a partir dali.

Nenhum arquivo novo entra hoje. Os métodos são novos dentro do `personagem.py` que já existe, e a chamada nova é uma linha dentro do laço que já existe no `main.py`.

melhorar_ataque

Crie o método `melhorar_ataque(self, quantidade)` em `Personagem`, no `personagem.py`.

O QUE ELE PRECISA TER

- Um parâmetro `quantidade`, além do `self`.
- Uma linha só, somando `quantidade` em `self.ataque`.
- Não imprime nada.

CHECKPOINT 1

O método `melhorar_ataque` existe em `Personagem`.

Chamar ele soma o valor certo em `ataque`, e não mexe em mais nada.

TESTE DO CHECKPOINT 1

```
h = Guerreiro("Thoric")
print("antes:", h.ataque)
h.melhorar_ataque(4)
print("depois:", h.ataque)
```

Resultado esperado:

```
antes: 18
depois: 22
```

Se o número não mudar: confira se o método está escrevendo `self.ataque`, com `self.` na frente.

EXERCÍCIO 2

melhorar_defesa

Agora o irmão gêmeo do exercício 1, pensando em `defesa` em vez de `ataque`.

O QUE ELE PRECISA TER

- Mesma forma de `melhorar_ataque`: um parâmetro `quantidade`.
- Uma linha só, somando `quantidade` em `self.defesa`.

CHECKPOINT 2

O método `melhorar_defesa` existe em `Personagem`.

Chamar ele soma o valor certo em `defesa` — não em `ataque`.

TESTE DO CHECKPOINT 2

```
h = Guerreiro("Thoric")
print("antes:", h.defesa)
h.melhorar_defesa(2)
print("depois:", h.defesa)
```

Resultado esperado:

```
antes: 8
depois: 10
```

Se o ataque mudou em vez da defesa: sobrou uma linha copiada de `melhorar_ataque` sem trocar o atributo.

melhorar_vida_maxima

Este é diferente dos dois primeiros: mexe em **dois** atributos, e os dois são privados.

O QUE ELE PRECISA TER

- Um parâmetro `quantidade`.
- Somar `quantidade` em `self._vida_maxima` — **com o** `_` na frente.
- Somar `quantidade` também em `self._vida` — também **com o** `_`.

ANTES DE RODAR

Se você escrever `self.vida_maxima` ou `self.vida`, sem o `_`, o Python vai recusar com um `AttributeError` dizendo que a property não tem setter. Isso não é um bug seu — é a classe se protegendo. Acrescente o `_` e o erro some.

CHECKPOINT 3

O método `melhorar_vida_maxima` existe em `Personagem`.

Chamar ele aumenta `vida_maxima` e `vida` no mesmo tanto.

TESTE DO CHECKPOINT 3

```
h = Guerreiro("Thoric")
h.receber_dano(40)
print("antes:", h.vida, "/", h.vida_maxima)
h.melhorar_vida_maxima(20)
print("depois:", h.vida, "/", h.vida_maxima)
```

Resultado esperado:

```
antes: 88 / 120
depois: 108 / 140
```

Se der `AttributeError: ... has no setter`: falta o `_` em uma das duas linhas do método.

subir_nivel

Agora junte os três métodos anteriores em um só, que decide os valores de uma vez.

O QUE ELE PRECISA TER

- No `__init__` de `Personagem`, acrescente `self.nivel = 1`.
- `subir_nivel(self)` — sem parâmetros além do `self`.
- Soma 1 em `self.nivel`.
- Chama `self.melhorar_ataque(4)`, `self.melhorar_defesa(2)` e `self.melhorar_vida_maxima(20)`.
- Imprime uma mensagem avisando o nível novo.

CHECKPOINT 4

O método `subir_nivel` existe em `Personagem` e chama os três métodos dos exercícios 1, 2 e 3 — ele não soma nada nos atributos diretamente.

`self.nivel` começa em 1 e sobe 1 a cada chamada.

TESTE DO CHECKPOINT 4

```
h = Guerreiro("Thoric")
h.subir_nivel()
h.subir_nivel()
print("nível:", h.nivel, "| ataque:", h.ataque,
      "| defesa:", h.defesa, "| vida máxima:", h.vida_maxima)
```

Resultado esperado:

```
Thoric subiu para o nível 2
Thoric subiu para o nível 3
nível: 3 | ataque: 26 | defesa: 12 | vida máxima: 160
```

Se `nivel` não existir: falta a linha `self.nivel = 1` no `__init__`.

EXERCÍCIO 5

Chamar no jogo

Os métodos só valem alguma coisa se o jogo realmente os chamar. No `main.py`, dentro do laço `for fase in fases:`, acrescente a chamada logo depois de confirmar a vitória.

```
venceu = combate(jogador, fase.inimigo)
if not venceu:
    print("Fim de jogo.")
    return

jogador.subir_nivel()

jogador.curar(50)
...
```

CHECKPOINT 5, QUASE O ÚLTIMO

`jogador.subir_nivel()` é chamado uma vez para cada fase vencida, antes do `jogador.curar(50)`.

O jogo roda do início ao fim sem erro na tela.

TESTE FINAL

Jogue até vencer a primeira fase. Confira se aparece na tela:

```
Thorric subiu para o nível 2
```

E se o próximo `mostrar_status()`, durante o combate seguinte, mostra a vida e o ataque já maiores que no começo.

Mostrar o nível

Um retoque pequeno para fechar a aula: `mostrar_status` ainda não fala do nível. Hoje ele fala.

O método hoje é este:

```
def mostrar_status(self):
    print("\n", self.nome)
    print("Vida:", self._vida, "/", self._vida_maxima)
    print("Mochila:", self.inventario.quantidade(), "item(ns)")
```

O QUE MUDA

- Só a primeira linha do `print`. As outras duas ficam exatamente como estão.
- Acrescente o nível ao lado do nome — o formato exato é com você.

CHECKPOINT 6, O ÚLTIMO

`mostrar_status` mostra o nível atual do personagem, além do que já mostrava.

TESTE DO CHECKPOINT 6

```
h = Guerreiro("Thoric")
h.subir_nivel()
h.mostrar_status()
```

Resultado esperado (o formato exato do texto pode variar, o número não):

```
Thoric - Nível 2
Vida: 140 / 140
Mochila: 2 item(ns)
```

Antes de encerrar, confira

- ☐ `melhorar_ataque`, `melhorar_defesa` e `melhorar_vida_maxima` existem em `personagem.py`
- ☐ Cada um mexe só no que é dele — nenhum atalho tocando dois atributos de uma vez, fora `melhorar_vida_maxima`
- ☐ `subir_nivel` chama os três métodos acima, não repete a soma

- ☐ `self.nivel` começa em 1 e aparece em `mostrar_status`
- ☐ `main.py` chama `jogador.subir_nivel()` uma vez por fase vencida
- ☐ O jogo roda do início ao fim sem erro na tela